

Algorithmic Complexity Analysis

Railway Station Search System

1. Introduction

This document presents a detailed analysis of the time complexity of the algorithms implemented in the railway station search system using KD-Tree (K-Dimensional Tree) data structures.

2. RangeSearch.java

2.1 Main Search Method

The `search` method implements a range search in a KD-Tree, allowing the retrieval of all stations within a specific geographic area.

Analyzed Code:

```
private HashMap<String, RailwayStation> search(
    KNode node,
    Range range,
    Boolean level,
    RailwayStationSearchFilters filters,
    HashMap<String, RailwayStation> resultStations) {

    if (node == null) return resultStations;

    if (range.contains(node.getLatitude(), node.getLongitude())) {
        for(RailwayStation rS : node.getStations()){
            if(filters.respectsFilters(rS)){
                resultStations.put(rS.getName(), rS);
            }
        }
    }

    double cord = level ? node.getLongitude() : node.getLatitude();
    double max = level ? range.getMaxLon() : range.getMaxLat();
    double min = level ? range.getMinLon() : range.getMinLat();

    if (cord >= min && node.getLeft() != null)
        search(node.getLeft(), range, !level, filters, resultStations);
    if (cord <= max && node.getRight() != null)
        search(node.getRight(), range, !level, filters, resultStations);

    return resultStations;
}
```

2.2 Time Complexity Analysis

Algorithm Operations:

Step 1: Base case verification

```
if (node == null) return resultStations;
```

- Complexity: $O(1)$

Step 2: Range containment check

```
if (range.contains(node.getLatitude(), node.getLongitude()))
```

- **Complexity:** $O(1)$

Step 3: Station processing

```
for(RailwayStation rS : node.getStations()){
    if(filters.respectsFilters(rS)){
        resultStations.put(rS.getName(), rS);
    }
}
```

- **k** = number of stations per node
- `respectsFilters(rS) :  $O(1)$`
- `HashMap.put() :  $O(1)$  amortized`
- **Complexity:** $O(k)$

Step 4: Recursive calls

```
if (cord >= min && node.getLeft() != null)
    search(node.getLeft(), range, !level, filters, resultStations);
if (cord <= max && node.getRight() != null)
    search(node.getRight(), range, !level, filters, resultStations);
```

- **Complexity:** Depends on tree structure and range

2.3 Worst Case Analysis

Time Complexity: $T(n) = O(n \cdot k)$

Conditions:

- The search range covers the entire geographic area
- All nodes must be visited
- Both recursive calls execute at each level

Mathematical Analysis:

The recurrence relation is:

$$T(n) = T(n_{\text{left}}) + T(n_{\text{right}}) + O(k)$$

In worst case, all n nodes are visited:

$$\begin{aligned} T(n) &= T(n-1) + O(k) \\ &= n \times O(k) \\ &= O(n \cdot k) \end{aligned}$$

Explanation:

- Every node in the tree is examined
- For each node, k stations are processed
- No pruning occurs in the tree traversal
- Total operations proportional to total number of stations

2.4 General Case Analysis

Time Complexity: $T(n) = O(\sqrt{n} + m \cdot k)$

For balanced KD-Trees in d -dimensional space:

The expected complexity of range queries is $O(n^{1-1/d} + m)$, where:

- **d** = number of dimensions (2 for latitude/longitude)
- **m** = number of nodes within the range
- **k** = average stations per node

For 2D case ($d=2$):

$$\begin{aligned}
 T(n) &= O(n^{(1-1/2)} + m \cdot k) \\
 &= O(n^{(1/2)} + m \cdot k) \\
 &= O(\sqrt{n} + m \cdot k)
 \end{aligned}$$

Analysis Components:

1. Tree Traversal: $O(\sqrt{n})$

- The algorithm examines approximately $\sqrt{n}$ nodes during traversal
- This represents the cost of navigating the KD-Tree structure
- Nodes examined but not necessarily within range

2. Station Processing: $O(m \cdot k)$

- **m** nodes fall within the specified range
- Each node contains **k** stations to process
- Filter check and HashMap insertion per station: $O(1)$

Mathematical Justification:

$$\begin{aligned}
 \text{Total Time} &= (\text{Traversal Cost}) + (\text{Processing Cost}) \\
 &= O(\sqrt{n}) + O(m \cdot k) \\
 &= O(\sqrt{n} + m \cdot k)
 \end{aligned}$$

Dependency on Range Size:

The value of **m** directly impacts performance:

- **Small range:** $m \ll n \rightarrow \text{Complexity} \approx O(\sqrt{n})$
- **Medium range:** $m \approx \sqrt{n} \rightarrow \text{Complexity} \approx O(\sqrt{n} \cdot (1+k))$
- **Large range:** $m \approx n \rightarrow \text{Complexity} \approx O(n \cdot k)$

Key Characteristics:

- Sublinear in tree size for selective queries
- Most practical queries involve small ranges (small **m**)
- Significantly faster than worst case for typical usage
- Performance degrades gracefully as range size increases

3. RangeSearchRepository.java

3.1 Repository Structure

```
private HashMap<Range, HashMap<RailwayStationSearchFilters, RangeSearch>> rangeSearchMap;
```

The repository implements a two-level cache using nested HashMaps to store previous search results.

3.2 rangeSearchExist Method

```
public Boolean rangeSearchExist(RailwayStationSearchFilters filters, Range range) {
    return rangeSearchMap.containsKey(range) &&
        rangeSearchMap.get(range).containsKey(filters);
}
```

Time Complexity Analysis:

General Case: $O(1)$

- `containsKey(range)`: $O(1)$ average
- `get(range)`: $O(1)$ average
- `containsKey(filters)`: $O(1)$ average
- Total: $O(1)$ amortized

Worst Case: $O(R + F)$

- HashMap collision chains degrade performance
- **R** = number of ranges in outer map
- **F** = number of filters in inner map
- Requires proper `hashCode()` and `equals()` implementation

3.3 addRangeSearch Method

```
public void addRangeSearch(RailwayStationSearchFilters filters,
                          RangeSearch rangeSearch,
                          Range range) {
    if (!rangeSearchMap.containsKey(range)) {
        rangeSearchMap.put(range, new HashMap<>());
    }
    rangeSearchMap.get(range).put(filters, rangeSearch);
}
```

Time Complexity Analysis:

Operations:

- containsKey(range) : O(1) average
- put(range, new HashMap<>()) : O(1) average
- get(range) : O(1) average
- put(filters, rangeSearch) : O(1) average

General Case: O(1)

Worst Case: O(R + F)

- HashMap resizing may occur
- Hash collision resolution

4. Complexity Summary

4.1 RangeSearch Algorithm

Operation	General Case	Worst Case
search()	$O(\sqrt{n} + m \cdot k)$	$O(n \cdot k)$

Where:

- n = total number of nodes in KD-Tree
- m = number of nodes within search range
- k = average number of stations per node

4.2 Repository Operations

Operation	General Case	Worst Case
rangeSearchExist()	O(1)	O(R + F)
addRangeSearch()	O(1)	O(R + F)

Where:

- R = number of cached ranges
 - F = number of cached filters per range
-